

## **CSS** Selectors

29

### **Selectors**

- CSS selectors select the HTML element(s) to style
- There are five families of selectors:
  - Basic
  - Combinator
  - Pseudo-class
  - Pseudo-element
  - Attribute

30

## Basic selectors

### Element selector

```
p {text-align: center; color: red;}
```

### Id selector

```
#parag1 {text-align: center;
          color: red;
        }
```

### Universal selector

```
* {
  text-align: center;
  color: red;
}
```

### class selector

```
.center {
  text-align: center;
  color: red;
}
```

### Grouping selector

```
h1, h2, p {
  text-align: center;
  color: red;
}
```

31

## Basic selectors: id selector

```
<!DOCTYPE html>
<html>
<head>
<style>
#parag1 {
  text-align: center;
  color: blue;
}
</style>
</head>
<body>

<p id="parag1">Hello, I am a blue
paragraph</p>
<p>I am a blue paragraph.</p>

</body>
</html>
```

I am a blue paragraph.

Hello, I am a blue paragraph

32

## Basic selectors: class selector

```
<!DOCTYPE html>
<html>
<head>
<style>
  .center {
    text-align: center;
    color: red;
  }
</style>
</head>
<body>
```

```
<h1 class="center">This heading is red
and centered</h1>
<p class="center">This paragraph is red
and centered</p>

</body>
</html>
```

33

**This heading is not red and centered**

This paragraph is red and centered

## Basic selectors: universal selector

```
<!DOCTYPE html>
<html>
<head>
<style>
  * {
    text-align: center;
    color: blue;
  }
</style>
</head>
<body>
```

```
<h1>I am a blue centered title</h1>

<p>I am blue and centered too!.</p>
<p id="para1">Me too!</p>
<p>And me again!</p>

</body>
</html>
```

34

**I am a blue centered title**

I am blue and centered too!.

Me too!

And me again!

## Basic selectors: grouping selector

```
<!DOCTYPE html>
<html>
<head>
<style>
h1, h2, p {
  text-align: center;
  color: red;
}
</style>
</head>
<body>

<h1>I am red</h1>
<h2>me too</h2>
<p>and me too.</p>

</body>
</html>
```

**I am red**

**me too**

and me too.

35

## Combinator

- Combinators select elements exploiting the specific relationship between them
- There are five different combinators in CSS:
  - Simple + class (.)
  - descendant selector (space)
  - child selector (>)
  - adjacent sibling selector (+)
  - general sibling selector (~)

Combinators sono simboli! Togli sele

```
p.red {color: red; text-align: center;}
```

```
div p {color: red;}
```

```
div > p {color: red;}
```

```
div + p {color: red;}
```

```
div ~ p { color: red;}
```

36

## Adjacent sibling and General sibling

```
<div>
  <p>I am a paragraph 1 in this div</p>
  <p>I am paragraph 2 in this div</p>
</div>

<p>I am paragraph 3 after the previous div</p>
<p>I am paragraph 4 after the previous div</p>
```

```
<div>
  <p> I am a paragraph </p>
</div>

<p> I am a paragraph </p>
<code> Here there is some code </code>
<p> I am another paragraph </p>
```

37

## Pseudo-class and Pseudo elements selectors

- Pseudo-class selectors are used to define a special state of an element: mouse over, link is visited or not and so on
- Pseudo-element selectors are used to style specified parts of an element: the first letter, the first line, and so on

38

## Attribute selectors

| Selector            | Example              | Description                                                                             |
|---------------------|----------------------|-----------------------------------------------------------------------------------------|
| [attribute]         | [target]             | Selects all elements with a target attribute                                            |
| [attribute=value]   | [target=_blank]      | Selects all elements with target="_blank"                                               |
| [attribute~=value]  | [title~=flower]      | Selects all elements with a title attribute containing the word "flower"                |
| [attribute =value]  | [lang=en]            | Selects all elements with a lang attribute value equal to "en" or starting with "en-"   |
| [attribute^=value]  | a[href^="https"]     | Selects every <a> element whose href attribute value begins with "https"                |
| [attribute\$=value] | a[href\$=".pdf"]     | Selects every <a> element whose href attribute value ends with ".pdf"                   |
| [attribute*=value]  | a[href*="w3schools"] | Selects every <a> element whose href attribute value contains the substring "w3schools" |

39

## Example

```

<!DOCTYPE html>
<html>
<head>
<style>
[title~= "flower"] {
border: 5px solid yellow;
}
</style>
</head>
<body>

<h2>CSS [attribute~= "value"] Selector</h2>
<p>All images with the title attribute containing the word "flower" get a
yellow border.</p>





</body>
</html>

```

### CSS [attribute~= "value"] Selector

All images with the title attribute containing the word "flower" get a yellow border.



[https://www.w3schools.com/css/tryit.asp?filename=trycss\\_sel\\_attribute\\_value2](https://www.w3schools.com/css/tryit.asp?filename=trycss_sel_attribute_value2)

40

# CSS

## Cascading and positioning

41

### Something unexpected can happen...

- Sometimes the style you thought should be applied to some elements is not appearing...
- Conflicting rules!

```
<button class="branding">Hello there!</button>
```

```
button { color: red; }  
.branding { color: blue; }
```

Let's see some hints!

42

## Cascading algorithm

- The steps that apply to the cascading order:

- Selection of rules
- Origin and importance
- Specificity
- Order of appearance

- Origin and importance:

| Order (high to low) | Origin               | Importance |
|---------------------|----------------------|------------|
| 1                   | user-agent (browser) | !important |
| 2                   | user                 | !important |
| 3                   | author (developer)   | !important |
| 4                   | author (developer)   | normal     |
| 5                   | user                 | normal     |
| 6                   | user-agent (browser) | normal     |

43

## Cascading algorithm

- Specificity: In case of equality with an origin, the specificity of a rule is the choice criterium. The declaration with the highest specificity wins.
- Order of decreasing specificity:
  - Id selectors
  - Class selectors / Attribute selectors / Pseudo-class selectors
  - Element selectors
  - Universal selector DOES NOT impact the specificity
  - Combinator selectors +, >, ~ DO NOT impact the specificity
- Inline styles (inline CSS) always overwrite any normal style in author stylesheets, so they have the highest specificity. The only way to override an inline style is to use the flag !important.

44



## Cascading algorithm

- Specificity algorithm (scoring system):

| inline                                                                            | ID                                                                                | CLASS                                                                                 | ELEMENT                                                                             |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|  |  |      |  |
| Add 1                                                                             | Add 1 here every time there is an ID selector in the rule                         | Add 1 here every time there is a class /attribute / pseudo-class selector in the rule | Add 1 here every time there is an element selector in the rule                      |



div { color: red; } /\* 0 - 0 - 1 \*/

#div { color: blue; } /\* 1 - 0 - 0 \*/

div.nav { color: lime; } /\* 0 - 1 - 1 \*/

The `!important` declaration does not add any points to the specificity. It only modifies the application priority of a rule, making it override other conflicting rules, regardless of specificity.

45

## Cascading algorithm

- Order of appearance: In the origin with precedence, if there are competing values for a property that are in style block matching selectors of equal specificity, the last declaration in the style order is applied.

p {color: black;}

p {color: red;}

46



## Quiz time!

47

### Quiz 1

```
.parent{  
  color:blue;  
  background-color: green;  
  width: 500px;  
  height: 500px; }
```

```
.box4.box{  
  color: yellow;  
  font-size:1em; }
```

```
#b {color: pink; }
```

```
div > .box{  
  font-size: 3em;  
  background-color:aqua;  
  border: 2px solid black;  
  margin: 10px; }
```

```
<body>  
  <div class="parent">  
    <div class="box" style="color:purple;">I am a box</div>  
    <div class="box" id="a">I am a box too</div>  
    <div class="box">Same here!</div>  
    <div class="box box4" id="b">Hi bros!</div>  
  </div>  
</body>
```

48

## Inheritance

- Another relevant concept is inheritance:  
In CSS, some styles are automatically “passed down” from parent elements to child elements

Text properties (e.g. color, font-family) are usually inherited automatically  
Layout properties (e.g. padding, margin, border) are not inherited automatically

49

## Quiz 2

```
<!DOCTYPE html>
<html>
<head>
  <style>
    span+p {color: red}
    span.p{color: green}
    body{color: orange}
    .p{color: purple}
  </style>
</head>
<body>
  <span class="p">A</span>
  <span>B</span> <p>C</p>
  <span class="p">D</span> <p class="p">E</p>
</body>
</html>
```

50

## Quiz 3

- Determine the specificity of the following rules:

```
#main .sidebar div {fontsize: 16px;}
div > .container {color: green;}
div ~ .item {color: red;}
div * {margin: 10px;}
```

Indicate which rule has the greatest specificity

51

## Positioning

- The **position** property is used to set position for an element. Only after position is set, the **left**, **right**, **top**, and **bottom** properties can be used. These properties behave differently, depending on the value of position
- There are several position values:

**Static**



The default position of all the HTML elements. They are always positioned according to the normal flow of the page

**Relative**



Setting the top, right, bottom, and left properties of a relatively-positioned element will cause it to be adjusted away from its normal position

**Absolute**



Setting the top, right, bottom, and left properties of a absolutely-positioned element will cause it to be adjusted in an absolute position with respect to its nearest positioned ancestor (if there is not ancestor, use the body)

52

## **Much more!**

- Pagination
- Multiple column
- Tooltips
- Graphic Transformations (e.g. 2D, 3D rotations)
- Animations

53

## **Cool! But complex, is not?**

- When websites are made of many pages, developing everything with HTML and CSS becomes very time consuming!
- Moreover, it is complicate to build websites working across browsers and that are responsive (i.e. that can adapt to different screen's sizes)

Why not to use frontend toolkits!

54

## Bootstrap



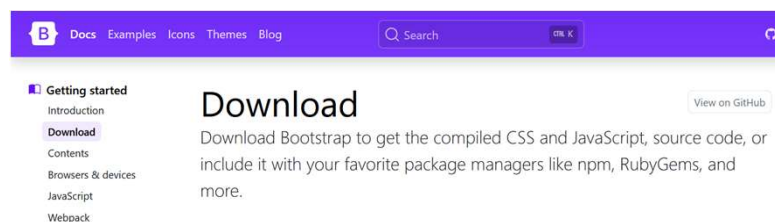
- Bootstrap was created in 2010 by Twitter developers and released under an open-source license
- The latest version is Bootstrap **version 5.x**
- Bootstrap is composed by:
  - HTML and CSS templates for elements such as typography, tables, forms, images and so on
  - JS plugins

55

## Installing Bootstrap

- There are several ways to install Bootstrap  
Look at:

<https://getbootstrap.com/docs/5.3/getting-started/download/>



Suggestion: the fastest way is to include it from a CDN  
(Content Delivery Network)

56

## Bootstrap basics: containers

- Containers are used to organize the content of the website

```
<div class="container" style="height: 550px; background-color: blue">
</div>
```

- They are layout element that provides a centered, responsive fixed-width or full-width wrapper for page content. They automatically applies horizontal padding and ensures content is properly aligned across different screen sizes

57

## Bootstrap basics: containers

- They can automatically adjust their width to the width of the screens through **breakpoints**:

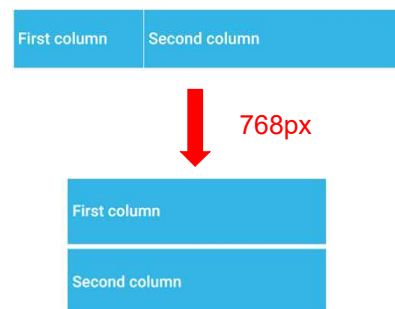
|                               | Extra small<br><576px | Small<br>≥576px | Medium<br>≥768px | Large<br>≥992px | X-Large<br>≥1200px | XX-Large<br>≥1400px |
|-------------------------------|-----------------------|-----------------|------------------|-----------------|--------------------|---------------------|
| <code>.container</code>       | 100%                  | 540px           | 720px            | 960px           | 1140px             | 1320px              |
| <code>.container-sm</code>    | 100%                  | 540px           | 720px            | 960px           | 1140px             | 1320px              |
| <code>.container-md</code>    | 100%                  | 100%            | 720px            | 960px           | 1140px             | 1320px              |
| <code>.container-lg</code>    | 100%                  | 100%            | 100%             | 960px           | 1140px             | 1320px              |
| <code>.container-xl</code>    | 100%                  | 100%            | 100%             | 100%            | 1140px             | 1320px              |
| <code>.container-xxl</code>   | 100%                  | 100%            | 100%             | 100%            | 100%               | 1320px              |
| <code>.container-fluid</code> | 100%                  | 100%            | 100%             | 100%            | 100%               | 100%                |

58

## Bootstrap basics: grid systems

- Grid systems are used for creating page layouts using rows and columns that house the content
- A grid is rendered as a container having rows that wrap columns. A grid houses up to 12 columns in one line. If there is more than 12 columns, the excess column will jump to the next line

```
<div class="container">
  <div class="row">
    <div class="col-md-4">First column</div>
    <div class="col-md-8">Second column</div>
  </div>
</div>
```



59

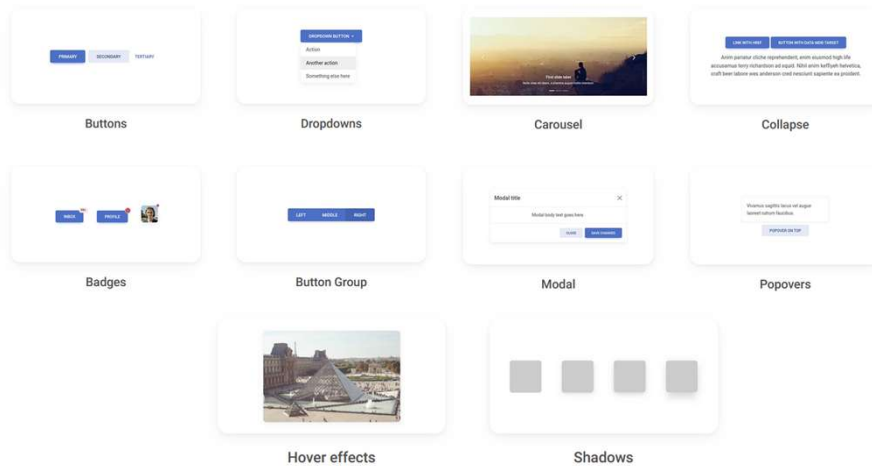
## Bootstrap basics: grid system

| Breakpoint        | Class infix | Dimensions |
|-------------------|-------------|------------|
| X-Small           | <i>None</i> | 0–576px    |
| Small             | <i>sm</i>   | ≥576px     |
| Medium            | <i>md</i>   | ≥768px     |
| Large             | <i>lg</i>   | ≥992px     |
| Extra large       | <i>xl</i>   | ≥1200px    |
| Extra extra large | <i>xxl</i>  | ≥1400px    |

60



## Bootstrap components



...and much more!

61

## Practicing @home

- Now, it is up to you!
- Restyle your restaurant website using CSS / Bootstrap using:
  - a navbar displayed in every page
  - a carousel in the home page
  - 3 cards describing different types of menu in the menu page
  - a form in the contact page
  - a component of your choice!

62